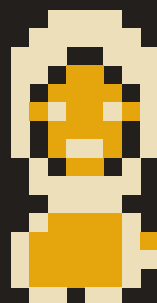




# SYNCPATTERNS

// caçando padrões com Helena e Marcus



VS



## Descobrimdo padrões

A jornada para sincronizar dados complexos.

Todas as Maiores Subsequências Comuns distintas, com Programação Dinâmica e Backtracking.

Fundamentos de Projeto e Análise de Algoritmos. PUC Minas Contagem. Profa. Amália Vasconcelos.

Daniel Heringer, Lucas Borges de Freitas, Mateus Botelho de Souza, Victor Alves Alcântara, Vitor Mendonça Braga, Iago Gonçalves Moysés.

# Duas fontes, o mesmo fenômeno

Helena e Marcus analisam grandes volumes de eventos: logs de servidores, transações financeiras, sensores de uma fábrica. Cada um registra a própria sequência, com formatos diferentes.

Para confiar nos dados, eles precisam **sincronizar** as fontes: achar os padrões comuns, sempre **respeitando a ordem** dos eventos.



HELENA

fonte 1 . ijkijkii



MARCUS

fonte 2 . ikjikji

# Todas as maiores subsequências comuns

Uma **subsequência** mantém a ordem mas pode pular posições. A **LCS** é a maior cadeia presente, na mesma ordem, nas duas fontes.

O ponto central: pode existir **mais de uma** LCS de tamanho máximo. Queremos **todas**, distintas e em ordem alfabética.

## ENTRADA

D conjuntos (D até 10). Cada um traz a sequência de Helena e a de Marcus, com 1 a 80 letras de a a z.

## SAÍDA

Todas as LCS distintas, ordenadas, sem repetir.  
Linha em branco entre conjuntos.

## Duas formas de resolver

A força bruta testaria  $2^n$  subsequências (inviável para  $n$  até 80). A ideia foi separar o problema: a PD calcula o tamanho ótimo, e o backtracking lista todas as soluções que atingem esse tamanho.

### ABORDAGEM 1

#### Somente Programação Dinâmica

- ▶ Cada célula guarda o conjunto de todas as LCS dos prefixos.
- ▶ As múltiplas soluções surgem da união nos empates.
- ▶ Sem recursão, só tabulação.

`lcs_dp.py`

### ABORDAGEM 2

#### Prog. Dinâmica + Backtracking

- ▶ PD constrói só a tabela de comprimentos.
- ▶ O backtracking reconstrói cada LCS pela tabela.
- ▶ Memoização evita recálculo.

`lcs_dp_backtracking.py`

# Programação Dinâmica

`dp[i][j]` guarda o tamanho da LCS dos prefixos de Helena e Marcus.

- ▶ casam: `dp[i][j] = dp[i-1][j-1] + 1`
- ▶ senão: `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`
- ▶ subestrutura ótima e subproblemas repetidos.

Custo da tabela de tamanhos: ordem de  $m$  vezes  $n$ .

|   |   | i | j | k |
|---|---|---|---|---|
|   | 0 | 0 | 0 | 0 |
| i | 0 | 1 | 1 | 1 |
| k | 0 | 1 | 1 | 2 |
| j | 0 | 1 | 2 | 2 |

## Backtracking

A tabela diz o tamanho, mas não quais são as LCS. Voltamos do canto até a origem. Quando os comprimentos **empatam**, o caminho se **ramifica**, e cada ramo pode revelar uma LCS diferente.

Sem backtracking acharíamos só uma. Com ele, enumeramos todas. No exemplo do roteiro, a ramificação gera **7 LCS** distintas de tamanho 5.

i j i j i

i j i k i

i j k j i

i k i j i

i k i k i

i k j i i

i k j k i



# Complexidade

$m, n$ : tamanhos .  $L$ : tamanho da LCS .  $R$ : nº de LCS distintas  
.  $K$ : maior conjunto mantido.

## SOMENTE PD

- ▶ tabela de tamanhos: ordem de  $m$  vezes  $n$ .
- ▶ conjuntos em cada célula: custo  $K$  vezes  $L$ .
- ▶ mais memória (guarda tudo na tabela).

$$O(m \cdot n \cdot K \cdot L)$$

## PD + BACKTRACKING

- ▶ tabela de tamanhos: ordem de  $m$  vezes  $n$ .
- ▶  $O(m \cdot n)$  estados, cada um uma vez.
- ▶ conjuntos sob demanda: menos memória.

$$O(m \cdot n) + O(m \cdot n \cdot K \cdot L)$$

# Mesma saída nas três versões

entrada

ijkijkii

ikjikji

7

LCS distintas

5

tamanho

saída (ordem alfabética)

ijiji

ijiki

ijkji

ikiji

ikiki

ikjii

ikjki

## O que aprendemos

- ▶ PD e backtracking se complementam: uma calcula o ótimo, o outro enumera.
- ▶ Empates na recorrência são a origem das múltiplas soluções.
- ▶ Memoização transforma busca exponencial em custo polinomial.
- ▶ Existe um trade-off claro entre tempo e memória.
- ▶ Validação, formatação e legibilidade também valem nota.

## Referências

- ▶ CORMEN, T. H. et al. Introduction to Algorithms. 3. ed. MIT Press, 2009.
- ▶ ZIVIANI, N. Projeto de Algoritmos com implementações em Java e C++. Cengage.
- ▶ Python docs, `functools.lru_cache`.  
[docs.python.org/3/library/functools.html](https://docs.python.org/3/library/functools.html)
- ▶ Pixelarticons (ícones, licença MIT). Fontes: Press Start 2P e VT323.

## Agora vamos jogar

Chegou a hora de abrir a interface gráfica e ver o algoritmo em ação: a matriz de PD se preenchendo e o backtracking ramificando até encontrar todas as LCS.



PRESS START

+3 pontos

arquivo: interface\_gráfica.html

**OBRIGADO. PERGUNTAS?**

Daniel Heringer, Lucas Borges de Freitas, Mateus Botelho de Souza, Victor Alves Alcântara, Vitor Mendonça Braga, Iago Gonçalves Moysés.

